

Efficient Range Deletes in LSM Engines

ABSTRACT

Modern data systems process enormous amount of data and selecting the appropriate data structure can have major implications in the responsiveness, availability and scalability of such systems. Traditional systems have certain limitations and thus more advanced Key-Value systems are used in most applications today. During updates, inserts and deletes, LSM trees utilize the main memory of the system to achieve better performance. Values are not deleted or updated in place. Each delete requires a new entry referred as a 'tombstone' that marks all previous entries with the same key as deleted. In our knowledge, none of the major Key-Value store developers have implemented range deletes except RocksDB. Their implementation use the so called 'range tombstones' which altogether consist a data-structure that can answer wheather a requested key is deleted by a range delete or not. For the database to construct such a response multiple I/Os are needed. Our method tries to reduce these I/Os by efficiently storing past results in memory in an opportunistic way, thus increasing responsiveness, since I/Os majorly affect performance.

1 INTRODUCTION

LSM-based Key-Value Stores. To store all information Key-Value stores use the log-structured merge (LSM) tree. LSM-trees store incoming data entries in main memory to facilitate writes, and periodically flush the entries as an immutable sorted run on durable storage. Runs store entries in a specific type of file called 'SST files'. During every compaction, the participating runs are sort-merged and invalidated entries are permanently deleted. Each compaction uses runs from consecutive levels. LSM-trees are adopted by several modern systems including LevelDB [1] and BigTable [2] at Google,

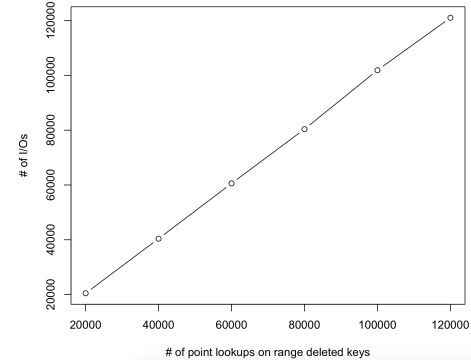


Figure 1: Many point-lookups on the same key increase I/Os linearly. PLACEHOLDER. ADD GRAPH WITH DELETES IMPACT INSTEAD. MAYBE KEEP THAT

RocksDB [3] at Facebook, X-Engine [4] at Alibaba, VolDEMORT [5] at LinkedIn, Dynamo [6] at Amazon, Cassandra [7], HBase [8], and Accumulo [9] at Apache, and bLSM [10] and cLSM [11] at Yahoo. MyRocks [12] uses RocksDB as storage engine and SQLite4 [13] has experimented with LSM-trees in its storage layer, while columnar systems use LSM-style updates [14].

Deletes in RocksDB. It might not seem obvious at first but deletes are probably the most common operation in LSM trees and RocksDB. Of course deletes are used to invalidate specific entries, but they are also used during updates. LSM trees use out-of-place updates which means that instead of updating the requested value, a new version is inserted in the higher levels. To complete this operation and avoid keeping older versions of values, LSM trees have to invalidate the older versions with a tombstone. As a result, the delete operation accounts for almost half of every update operation. This, combined with the fact that deletes happen very frequently in many workloads, suggest that both range and point deletes majorly affect the performance such systems. In this paper we will concentrate more on Range Deletes and ways to improve their performance.(MORE ON WHY RANGE DELETES IN SPECIFIC). When a range delete is issued, a range tombstone is generated and added to the database. A point request initiates a process where all range tombstones are fetched from the disk and fragmented in memory. This enables the system

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Under submission, June 14–19, 2020,

© 2023 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM... \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

to respond whether the requested key is deleted by such a range tombstone.

The Problem. As previously mentioned, if a key is deleted by a range delete, we use one I/O to fetch the key and its sequence number and then ask the range delete aggregator to check if it is deleted. Thus, even if the key is deleted, we waste one I/O. If the key is requested for a second time, the same operation takes place, wasting more I/Os linearly. This behavior can be seen in Figure 1.

The Solution. Our method reduces disk accesses when a key deleted by a range delete is accessed multiple times. It stores the deleted keys in a new key-value store which resides entirely in memory. This store contains deleted keys and gives no false-negative responses. In this way, when a range deleted key is requested the first time, it is saved in this KV store. All future requests will first ask this KV store before accessing the disk to obtain the sequence number, thus eliminating I/Os. In addition to that, we take advantage of the granularity that disk accesses happen. Instead of retrieving and storing information in our KV store for only the requested key, we store information about all the keys in the same page. In this way we can store more information and potentially avoid IOs on range deleted keys in the same page which may have not been requested yet.

2 LSM BACKGROUND

Basics. LSM-trees store data as key-value pairs. They have N levels and the first level, Level0, is stored in main memory and referred to as a buffer. The remaining $N-1$ levels are stored on external storage. RocksDB uses leveling instead of tiering which means that all levels consist of one run. (EXPLAIN MORE.) These $N-1$ levels store data in the SST files. SST stands for Sorted Sequence Table and they are persistent files storing data. In the file, keys are usually organized in sorted order so that a key or iterating position can be identified through a binary search. All new entries are initially stored in the buffer and are later compacted to lower (HIGHER?) levels. Compaction pushes keys and values to lower levels to free up space in memory. There are many compaction policies, some more aggressive and others not so much. This approach enables the system to treat updates, deletes and inserts very quickly as only the main memory is used. If an update or delete on a key is requested, the system first searches in the main buffer by first consulting a dedicated bloom filter, and if the key exists, it is in-place updated or permanently deleted. Otherwise, if the key is not in the buffer, instead of scanning the disk the system will insert a new entry in the tree. This entry has

absolutely no difference with a normal insert entry and will be eventually stored in an SST file. In the case of deleted the new entry has the same key as the one that needs to be deleted and a flag which showcases that it is a tombstone. This is an out-of-place delete as the value is not immediately deleted. Updates, work the same way, by inserting the newer version in memory but before that, a delete has to be initiated as the older version needs to be invalidated and later be deleted during compactions. The invalidated entries are eventually deleted when the file that contains the tombstone is compacted with the file that contains the value.

Partial Compaction. To amortize the latency spikes resulting from compactions in larger levels, state-of-the-art LSM based systems organize runs into smaller files, and perform compactions at the granularity of files instead of levels [22]. If Level i grows beyond a threshold capacity, a compaction is triggered and one file from Level i is chosen to be compacted with files from Level $i + 1$ that have an overlapping key-range, performing a partial compaction. Deciding which file to compact depends on the storage engine design. For instance, to optimize for high write throughput, we select the file from Level i that has minimal overlapping files in Level $i + 1$, to minimize both write amplification and compaction time. Partial compactions spread the total number of I/Os performed during compactions of an entire level over time to multiple smaller compactions.

Point deletes in RocksDB. Write something generic??

Point deletes read path implementation in RocksDB.

Deleting keys is a very common pattern in RocksDB. A point delete does not delete the actual value, but inserts a new entry to the LSM tree that is called "tombstone". Every tombstone has its own sequence number. The sequence number is an auto-incremented ID number that is used to implement snapshot read, garbage collection in compactions, MVCC in transactions, and some other purposes such as determining the age of the entries. The larger the sequence number, the newer the entry. When a point lookup is requested from the database, the newest entry is retrieved. If this entry is a tombstone it means that the key is deleted.

Range deletes read path implementation in RocksDB.

Most systems built on top of RocksDB have multi-component key schemas, where keys sharing a common prefix are logically related. The most common pattern is scan-and-delete, i.e., advance an iterator through the to-be-deleted range, and issue a delete for each key. This approach, even though functional majorly increases space amplification as it creates many tombstones, slows down

iterators and doesn't offer a deadline for space reclamation and increases I/Os. Not until recently, a dedicated mechanism for range deletes was implemented. However, even though it offers greater speeds in inserting delete ranges, it decreases look up speeds for non deleted keys. Also, the current system does not adapt and as a result executes 'wasted' I/Os in cases where a deleted key by a range delete is requested multiple times. Our method uses tries [?] in order to efficiently store keys that led to wasted I/Os and avoid making them in the future. The more the system operates, the more useful our tries become, thus saving I/Os which is the bound for most systems.

When a range delete is issued, it is first written to a dedicated memtable for range tombstones. As memtable we refer to the in-memory data structure that stores the most recent updates of the database.

Range tombstones. A range tombstone is composed of a start key, end key, and sequence number. Any key that falls within the range is considered deleted if the key's sequence number is less than or equal to the range tombstone's sequence number. Range tombstones are not fragmented in the memtable directly, but are instead fragmented each time a read occurs. Range tombstones are stored in the SST files themselves. Not in the data blocks, but in a separate meta block for range tombstones. This resolves the problem of when to obsolete range tombstones, as it's simple: when they're compacted to the bottom level. Each SST file contains all range tombstones at that level that cover user keys overlapping with the file's key range; this greatly simplifies iterator seeking and point lookups. When a new range delete is requested from the system, a range delete tombstone is created which is later applied to a dedicated range tombstone memtable. Later in the background the range tombstone memtable and its corresponding data memtable are flushed together into a single SST with a range tombstone meta-block. As new data is inserted, new compactions are executed. During these compactions, in the same way as with single key tombstones, obsolete range delete tombstones are permanently deleted, thus reducing storage occupancy.

Range deletes read path Implementation in RocksDB.

In a point lookup, RocksDB first looks at the fence pointers table kept in memory. This table keeps the page address and the first key of each page of the SST files. Then, by using binary search, the corresponding page is found. Afterwards, the bloom filter of each file probabilistically responds if the key is contained in the file. If not the key does not exist for sure. If yes, we execute an I/O to fetch the key and its sequence number. The next step is to determine if this key is deleted. If the key is deleted

by a point delete it means that the most recent version of that key in the LSM-tree is a tombstone, which is practically an entry with a flag to indicate that it is a delete. In a range delete however, when the requested key resides in a range deleted area, the fence pointer and bloom filter will bypass this range tombstone and lead to the SST file that the key is stored. Thus, there is no immediate way to decide whether the key is deleted.

Skyline vs RDA. In the the initial implementation of range deletes in RocksDB, each read operation of built up a 'skyline' for the range tombstones visible at the read sequence number. In point lookups, range tombstones were aggregated in an unordered vector, essentially the 'skyline' as they were search through live memtable, immutable memtables, and then SSTs. This Skyline approach can very quickly determine whether a key is deleted but is expensive to construct and maintain. The initial skyline range tombstone implementation has since been replaced with a more efficient lookup structure. The key change in the new implementation is to fragment the range tombstones that are stored in an sstable. The fragmented range tombstones provide the same benefit as the skyline representation: the ability to binary search the fragments in order to find the tombstone covering a key. But unlike the skyline approach, the fragmented tombstones can be cached on a per-sstable basis. In the v2 approach, a range delete aggregator (RDA) keeps track of the fragmented range tombstones for each sstable encountered during a read or iterator, and logically merges them together. In summary, in RocksDB the range delete aggregator acts as an oracle for answering whether a key is deleted at a particular sequence number. Due to caching of fragmented tombstones, the v2 implementation of RDA is significantly faster to populate than v1, yet the overall approach to processing range tombstones remains similar. The entire path is visible in figure 2

3 FACILITATING RANGE DELETES

Our Method. Our method minimizes wasted I/Os by efficiently storing in main memory the keys that have been determined as "deleted" from the range delete aggregator. We also check all other keys in the same page as the searched key. Since in every I/O we retrieve a page, checking these additional keys wastes minimal resources and at the same time benefits us as more useful information about range deleted keys is efficiently stored in memory. In a cold start, the LSM tree will have the exact same structure as in RocksDB. Each SST file will only have one Bloom Filter associated with, stored in main memory. If at some point, the database

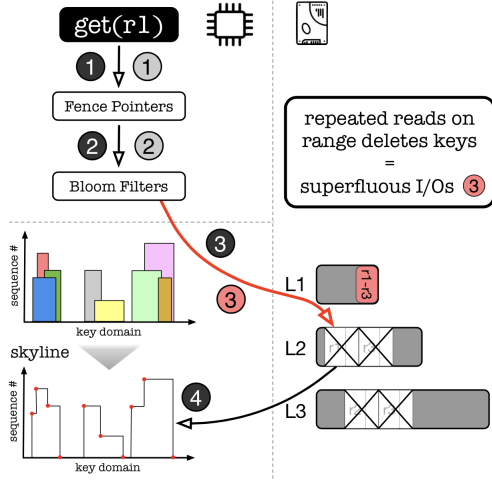


Figure 2: Read path.

receives a key lookup and the RDA responds that the key is deleted, then and only then, we construct a data structure called A that keeps the ranges of the deleted keys. When checking for a key, we also check for the other keys on the same page. Each SST file has its own A, which will eventually become more accurate as we have more key lookups for deleted keys in its different pages. In more detail, when the RDA responds that the key we are searching is deleted, we check all other keys in the same page. We might receive one of the results showing in Figure 4.

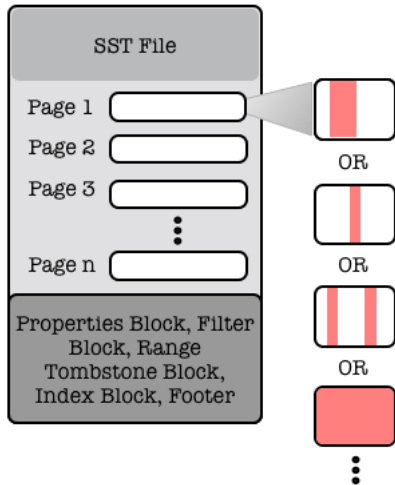


Figure 3: Possible results after requesting the deleted keys for a particular page from the RDA

In this example we suppose that we are looking for 'key3' and it is the first time that this key is looked up.

We will also assume that 'key3' is deleted. If not deleted, the I/O can not be avoided anyway. In the worst case (case2), 'key3' is the only key deleted in page. Thus, the resulted trie will only contain 'key3'. In the other cases, the A, will contain more than one key, even if these keys were never requested. Again, it is essential to mention that determining whether these keys are deleted is very quick by checking asking the RDA. No more than one I/O is used as one I/O retrieves exactly one page. Thus, we take advantage of all the data that a page contains.

Benefits of Maintaining an A. Again, we will assume that we are looking for a deleted key and that this key is not requested after it was deleted. As soon as this key is requested, we will have to execute one I/O and then check ask the RDA. Immediately after that, we will check all other keys in the same page and then construct an A which will be assigned to the corresponding SST file. From now on, this SST file will have both a Bloom filter and an A. If the key is requested for a second time, the Bloom filter of the file will respond positively that the key is in the file (with a certain probability), but the trie can guarantee that the key is deleted. As a result, a wasted I/O can be avoided. Also, because the A might contain other deleted keys of the same page, we benefit in case these keys are ever requested. The path can be seen in figure 4.

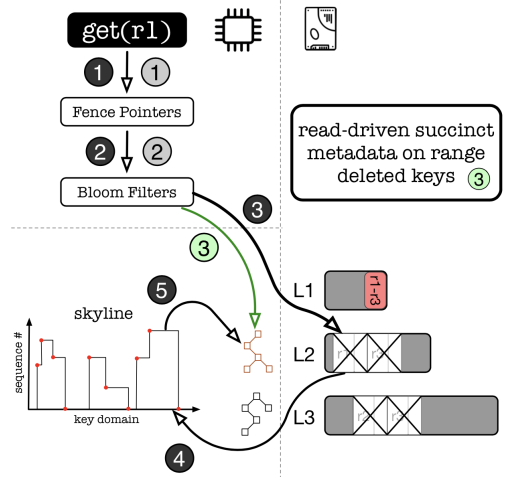


Figure 4: My path.

Dealing with Compactions. When an SST file merges with n files from a higher level, at most $n + 1$ files can be created. The A of the file in lower level certainly dominates the A of the files of the higher levels. Thus, a simple solution would be to keep the dominating A, split it accordingly, and then assign it to the files generated

after the merge. All other As can be discarded. However, we are still thinking of ways to retain as much useful data as we possibly can.

Challenges and Scope of Improvement. Our method uses extra memory to store the A in order to save I/Os. In a cold start it uses the same memory as RocksDB, however, if deleted keys are requested, new A are constructed, and more memory is used. In workloads, where there are many reads of deleted keys, our method might prove extremely beneficial as it saves I/Os. Our method is also read-driven. The results are totally depended on the workload. If the reads in the workload are sparse we might allocate a lot of memory with our As. Finally, read performance might suffer. If we assume that a key

A is not deleted by a range delete and that the SST file containing that key has a A, our method will first search the A, will determine that the key is not there, so it might not be deleted. The RDA will respond that the key is valid and the key will be returned. In this series of events, compared to RocksDB, our method had to check the existence of the key in the A. Although, our intuition is that this extra computation is inconsiderable, we are thinking of ways to minimize its impact.

4 EVALUATION

5 RELATED WORK

6 CONCLUSION